

Πανεπιστήμιο Ιωαννίνων

Τμήμα Πληροφορικής και Τηλεπικοινωνιών του Πανεπιστημίου Ιωαννίνων

Φίλτρα Bloom και Φίλτρα Bloom με Μετρητές

Αλγόριθμοι και Προχωρημένες Δομές Δεδομένων

1η Εργασία

ΜΑΡΙΑ ΑΛΕΞΑΝΔΡΑ ΝΤΟΝΤΟΥ

263

Περιεχόμενα

Θεωρητικό Πλαίσιο 3

Πιθανοτικές Δομές Δεδομένων 3

Φίλτρα Bloom 3

Bit Sets και Hash Functions 4

Μετρητές και Φίλτρα 4

Πλεονεκτήματα, Μειονεκτήματα και Περιπτώσεις Χρήσης 5

Υλοποίηση Κώδικα 5

Πειραματική Διαδικασία και Αποτελέσματα 8

Έλεγχος Ορθότητας – Μετρητές και Διαγραφή 8

Αξιολόγηση False Positives 9

Σύγκριση Απόδοσης με Python Set 9

Συμπεράσματα 11

Θεωρητικό Πλαίσιο

Η κατακόρυφη αύξηση του όγκου των πληροφοριών καθιστά συχνά απαγορευτική τη χρήση παραδοσιακών δομών αναζήτησης και αποθήκευσης, λόγω των αυστηρών περιορισμών στην κύρια μνήμη (RAM) των συστημάτων. Για να αντιμετωπιστεί αυτό το εμπόδιο, η αλγοριθμική σχεδίαση έχει στραφεί σε προσεγγιστικές τεχνικές, όπου η απόλυτη ακρίβεια θυσιάζεται ελεγχόμενα προς όφελος της βέλτιστης διαχείρισης των πόρων. Σε αυτό το πλαίσιο αναδεικνύονται οι δομές που αποτελούν τον πυρήνα της παρούσας εργασίας.

Πιθανοτικές Δομές Δεδομένων

Οι πιθανοτικές δομές δεδομένων παρέχουν μία λογική εκτίμηση της απάντησης και παρομοίως έναν τρόπο ώστε να προσεγγισθεί αυτή η εκτίμηση. Γενικά, σκοπός τους σε σχέση με τις στάνταρ δομές που προσπαθούν να κρατήσουν μια ισορροπία μεταξύ ταχύτητας εκτέλεσης και μνήμης, τις εξειδικευμένες που δίνουν προτεραιότητα στην ταχύτητα έναντι της μνήμης και των συμπιεσμένων που κάνουν το αντίθετο, είναι να καταναλώσουν όσο το δυνατό λιγότερη μνήμη διατηρώντας παράλληλα έναν εύλογο χρόνο εκτέλεσης.

Φίλτρα Bloom

Το Φίλτρο Bloom που επινοήθηκε από τον Burton H. Bloom το 1970 αποτελεί μια από τις πιο χαρακτηριστικές πιθανοτικές δομές, σχεδιασμένη ειδικά για να απαντά στο ερώτημα της συμμετοχής ενός στοιχείου σε ένα σύνολο (set membership). Η βασική του ιδιότητά είναι ότι ποτέ δεν επιστρέφει λανθασμένα αρνητικά αποτελέσματα (false negatives). Αν το φίλτρο αποφανθεί ότι ένα στοιχείο δεν υπάρχει, η απάντηση αυτή είναι ντετερμινιστικά ορθή. Αντίθετα, επιτρέπει ένα ελεγχόμενο ποσοστό λανθασμένων θετικών αποτελεσμάτων (false positives), απαντώντας δηλαδή ότι ένα στοιχείο «πιθανότατα υπάρχει» ενώ στην πραγματικότητα δεν έχει εισαχθεί ποτέ. Αυτός ο συμβιβασμός είναι που επιτρέπει στη δομή να επιτύχει εντυπωσιακή εξοικονόμηση μνήμης έναντι παραδοσιακών δομών αποθήκευσης [1].

Algorithm: Insert in Bloom filter.

`InsertInBloomFilter`(T, x)
Input: T, a bit array of size m
x, a record to insert to the set represented by the Bloom filter
Result: the record is inserted in the Bloom filter by setting the bits $h_0(x), h_1(x), \ldots, h_{k-1}(x)$ to 1

```
1  for i ← 0 to k do
2      h ← h_i(x)
3      T[h] ← 1
```

Algorithm: Check membership with Bloom filter.

`IsInBloomFilter`$(T, x) \rightarrow$ TRUE or FALSE
Input: T, a bit array of size m
x, a record to check if it is a member of the set represented by the Bloom filter
Output: TRUE if x is in the Bloom filter, FALSE otherwise

```
1  for i ← 0 to k do
2      h ← h_i(x)
3      if T[h] = 0 then
4          return FALSE
5  return TRUE
```

Εικόνα 1 Ψευδοκώδικας των βασικών λειτουργιών εισαγωγής (insert) και ελέγχου συμμετοχής (query) σε ένα κλασικό Φίλτρο Bloom. Via Εφαρμογές Κατακερματισμού, Καθηγητής Π. Λουρίδας ΟΠΑ.

Bit Sets και Hash Functions

Σε επίπεδο υλοποίησης, ένα κλασικό φίλτρο Bloom απαρτίζεται από έναν μονοδιάστατο πίνακα m-bits (bit set), τα οποία αρχικοποιούνται στην τιμή 0, καθώς και από ένα σύνολο k ανεξάρτητων hash functions. Κατά την εισαγωγή ενός νέου στοιχείου, αυτό τροφοδοτείται στις k συναρτήσεις, οι οποίες παράγουν h δείκτες (θέσεις) εντός των ορίων του πίνακα. Τα bits που αντιστοιχούν σε αυτές τις θέσεις τίθενται στην τιμή 1. Κατά τη διαδικασία ερωτήματος (query), το στοιχείο περνάει ξανά από τις ίδιες συναρτήσεις. Εάν έστω και ένα από τα παραγόμενα bits είναι 0, το στοιχείο απουσιάζει με απόλυτη βεβαιότητα. Αν, ωστόσο, όλα τα bits είναι 1, το σύστημα συμπεραίνει ότι το στοιχείο πιθανώς υπάρχει. Η πιθανότητα σφάλματος (false positive) προκύπτει επειδή τα συγκεκριμένα k bits ενδέχεται να έχουν τεθεί σε 1 από την επικάλυψη εγγραφών άλλων στοιχείων, και όχι από το ίδιο το στοιχείο που ελέγχεται.

Μετρητές και Φίλτρα

Ένας κρίσιμος περιορισμός του κλασικού φίλτρου Bloom είναι η αδυναμία διαγραφής στοιχείων. Εάν επιχειρούταν η διαγραφή ενός στοιχείου θέτοντας τα αντίστοιχα k bits του πάλι στο 0, θα υπήρχε άμεσος κίνδυνος αλλοίωσης της πληροφορίας κάποιου άλλου στοιχείου που έτυχε να χρησιμοποιήσει τα ίδια bits, παραβιάζοντας έτσι την εγγύηση απουσίας λανθασμένα αρνητικών αποτελεσμάτων. Το φίλτρο με μετρητές επιλύει αυτό το πρόβλημα αντικαθιστώντας τα απλά bits του πίνακα με μικρούς ακέραιους μετρητές. Στην περίπτωση αυτή, η εισαγωγή συνεπάγεται την αύξηση των αντίστοιχων μετρητών κατά 1, ενώ η αναζήτηση ελέγχει εάν όλοι οι μετρητές είναι αυστηρά μεγαλύτεροι του μηδενός. Έτσι, καθίσταται δυνατή η λειτουργία της

διαγραφής, η οποία απλώς μειώνει κατά 1 τους μετρητές που αντιστοιχούν στο στοιχείο, αποδεσμεύοντας τον χώρο χωρίς να επηρεάζονται τα υπόλοιπα, επικαλυπτόμενα στοιχεία του συνόλου [2].

Algorithm: Insert into counting Bloom filter.

`InsertInCntBloomFilter(`T, x`)`

Input: T, an integer array of size m
x, a record to insert to the set represented by the counting Bloom filter

Result: the record is inserted in the counting Bloom filter by increasing by 1 the counters at $h_0(x), h_1(x), \ldots, h_{k-1}(x)$

```
1 for i ← 0 to k do
2     h ← h_i(x)
3     T[h] ← T[h] + 1
```

Algorithm: Remove from a counting Bloom filter.

`RemoveFromCntBloomFilter(`T, x`)`

Input: T, an integer array of size m
x, a record to remove from the set represented by the counting Bloom filter

Result: the record is removed from the counting Bloom filter by decreasing by 1 the counters at $h_0(x), h_1(x), \ldots, h_{k-1}(x)$

```
1 for i ← 0 to k do
2     h ← h_i(x)
3     if T[h] ≠ 0 then
4         T[h] ← T[h] − 1
```

Εικόνα 2 Ψευδοκώδικας των δυναμικών λειτουργιών εισαγωγής (insert) και ασφαλούς διαγραφής (remove) σε ένα Φίλτρο Bloom με Μετρητές. Via Εφαρμογές Κατακερματισμού, Καθηγητής Π. Λουρίδας ΟΠΑ.

Πλεονεκτήματα, Μειονεκτήματα και Περιπτώσεις Χρήσης

Το κύριο πλεονέκτημα είναι η ακραία συμπίεση χώρου. Παράλληλα, εξασφαλίζει σταθερό χρόνο εκτέλεσης O(k) τόσο για την εισαγωγή όσο και για την αναζήτηση, ο οποίος είναι εντελώς ανεξάρτητος από το πλήθος των στοιχείων στη δομή αλλά και από το μέγεθος των ίδιων των δεδομένων. Ως μειονεκτήματα καταγράφονται η ύπαρξη λανθασμένα θετικών αποτελεσμάτων, η αδυναμία ανάκτησης/διάσχισης των αποθηκευμένων στοιχείων, καθώς και η ανάγκη γνώσης εκ των προτέρων της τάξης μεγέθους του συνόλου n, προκειμένου να μην υπερκορεστεί ο πίνακας και αυξηθεί δραματικά το ποσοστό σφάλματος. Οι δομές αυτές είναι αναντικατάστατες σε συστήματα όπου ο χώρος είναι περιορισμένος και η ταχύτητα κρίσιμη. Χρησιμοποιούνται ευρέως σε συστήματα Βάσεων Δεδομένων (όπως η Apache Cassandra και το PostgreSQL) για την αποφυγή δαπανηρών προσπελάσεων στον δίσκο όταν αναζητούνται εγγραφές που δεν υπάρχουν, στους Web Browsers για τον ταχύτατο έλεγχο αν ένα URL ανήκει σε λίστες κακόβουλων διευθύνσεων, καθώς και σε δρομολογητές δικτύων.

Υλοποίηση Κώδικα

Για την υλοποίηση της εργασίας αναπτύχθηκαν δύο βασικές κλάσεις η BloomFilter και η CountingBloomFilter, τα αντικείμενα των οποίων προσομοιώνουν τη λειτουργία των αντίστοιχων πιθανοτικών δομών. Η περιγραφή των χαρακτηριστικών, των βασικών μεθόδων

τους, καθώς και των βοηθητικών συναρτήσεων που περιλαμβάνονται στο αρχείο main.py, παρουσιάζεται αναλυτικά στον παρακάτω πίνακα.

Τμήμα Κώδικα /Συνάρτηση	**Περιγραφή**
class BloomFilter	Κλάση που προσομοιώνει τη λειτουργία του κλασικού πιθανοτικού φίλτρου Bloom, χρησιμοποιώντας έναν πίνακα μηδενικών (bit_array) μεγέθους m.
def _hash_functions(self, item)	Υλοποιεί τη μέθοδο του Double Hashing. Δέχεται ένα στοιχείο (string), υπολογίζει το MD5 hash του, το χωρίζει σε δύο δεκαεξαδικά τμήματα (h1, h2) και παράγει k μοναδικούς δείκτες μέσω της σχέσης (h1 + i * h2) * mod m.
BloomFilter.insert(self, item)	Δέχεται το στοιχείο προς εισαγωγή, παράγει τους k δείκτες από τη συνάρτηση κατακερματισμού και θέτει τις αντίστοιχες θέσεις του πίνακα σε 1 (True).
BloomFilter.query(self, item)	Ελέγχει την παρουσία ενός στοιχείου. Εάν έστω και μία από τις k θέσεις που υποδεικνύουν τα hashes είναι 0, επιστρέφει False. Αν όλες είναι 1, επιστρέφει True (πιθανό false positive).
class CountingBloomFilter	Κλάση που προσομοιώνει το φίλτρο Bloom με μετρητές. Αντί για bit array, χρησιμοποιεί

	πίνακα ακεραίων (counters) επιτρέποντας την αύξηση και μείωση (διαγραφή) τιμών.
CountingBloomFilter.insert	Για κάθε παραγόμενο δείκτη, αυξάνει τον μετρητή της αντίστοιχης θέσης κατά 1. Διαθέτει μηχανισμό ελέγχου υπερχείλισης (< 15), προσομοιώνοντας τους περιορισμούς ενός πραγματικού μετρητή 4-bit.
CountingBloomFilter.delete	Ελέγχει αρχικά εάν το στοιχείο υπάρχει (ώστε να αποφευχθεί η αλλοίωση ξένων μετρητών). Εφόσον υπάρχει, μειώνει τους αντίστοιχους μετρητές κατά 1.
get_random_string(length=8)	Βοηθητική συνάρτηση παραγωγής τυχαίων συμβολοσειρών, προκειμένου να παραχθεί ο όγκος δεδομένων (dataset) των 10.000 στοιχείων για τις δοκιμές απόδοσης.
visualize_results	Συνάρτηση οπτικοποίησης δεδομένων (μέσω matplotlib). Σχεδιάζει τρία υπο-γραφήματα: ένα ραβδόγραμμα για το ποσοστό των False Positives ανά μέγεθος πίνακα, ένα για τη σύγκριση χρόνου εκτέλεσης και ένα γράφημα πίτας για τη σύγκριση κατανάλωσης μνήμης.
main()	Η κύρια ροή του προγράμματος. Εκτελεί διαδοχικά: α) Έλεγχο ορθότητας λειτουργιών εισαγωγής/διαγραφής (με τις λέξεις "apple", "banana", "cherry"), β) το πείραμα των 10.000 εισαγωγών/1.000 ερωτημάτων για τον έλεγχο

	των σφαλμάτων (false positives) στα 3 μεγέθη πίνακα ($50k, 100k, 200k$), γ) τη συγκριτική αξιολόγηση χρόνου και μνήμης έναντι της δομής set της Python, και δ) την κλήση της οπτικοποίησης των αποτελεσμάτων.

Πειραματική Διαδικασία και Αποτελέσματα

Προκειμένου να αξιολογηθεί η λειτουργικότητα και η αποδοτικότητα των δομών που υλοποιήθηκαν, σχεδιάστηκε και εκτελέστηκε μια σειρά από πειράματα. Η διαδικασία αυτή διακρίνεται σε τρία βασικά στάδια: τον έλεγχο ορθότητας των λειτουργιών (εισαγωγή, αναζήτηση, διαγραφή), τη στατιστική μέτρηση των λανθασμένα θετικών αποτελεσμάτων (false positives) υπό φόρτο 10.000 στοιχείων για διάφορα μεγέθη πινάκων, και τέλος, τη συγκριτική ανάλυση χρόνου εκτέλεσης και κατανάλωσης μνήμης έναντι της εγγενούς δομής set της Python.

Έλεγχος Ορθότητας – Μετρητές και Διαγραφή

Για τον έλεγχο ορθότητας του Counting Bloom Filter, εισήχθησαν τα στοιχεία "apple", "banana" και "cherry". Σε αρχική αναζήτηση, το "apple" εντοπίστηκε επιτυχώς (True). Κατόπιν εκτέλεσης της συνάρτησης διαγραφής (delete("apple")), η δομή μείωσε σωστά τους αντίστοιχους μετρητές, με αποτέλεσμα η νέα αναζήτηση να επιστρέψει False. Παράλληλα, το "banana" συνέχισε να εντοπίζεται κανονικά, επιβεβαιώνοντας ότι η διαγραφή δεν επηρέασε επικαλυπτόμενα στοιχεία.Για τον έλεγχο ορθότητας του Counting Bloom Filter, εισήχθησαν τα στοιχεία "apple", "banana" και "cherry". Σε αρχική αναζήτηση, το "apple" εντοπίστηκε επιτυχώς (True). Κατόπιν εκτέλεσης της συνάρτησης διαγραφής (delete("apple")), η δομή μείωσε σωστά τους αντίστοιχους μετρητές, με αποτέλεσμα η νέα αναζήτηση να επιστρέψει False. Παράλληλα, το "banana" συνέχισε να εντοπίζεται κανονικά, επιβεβαιώνοντας ότι η διαγραφή δεν επηρέασε επικαλυπτόμενα στοιχεία.

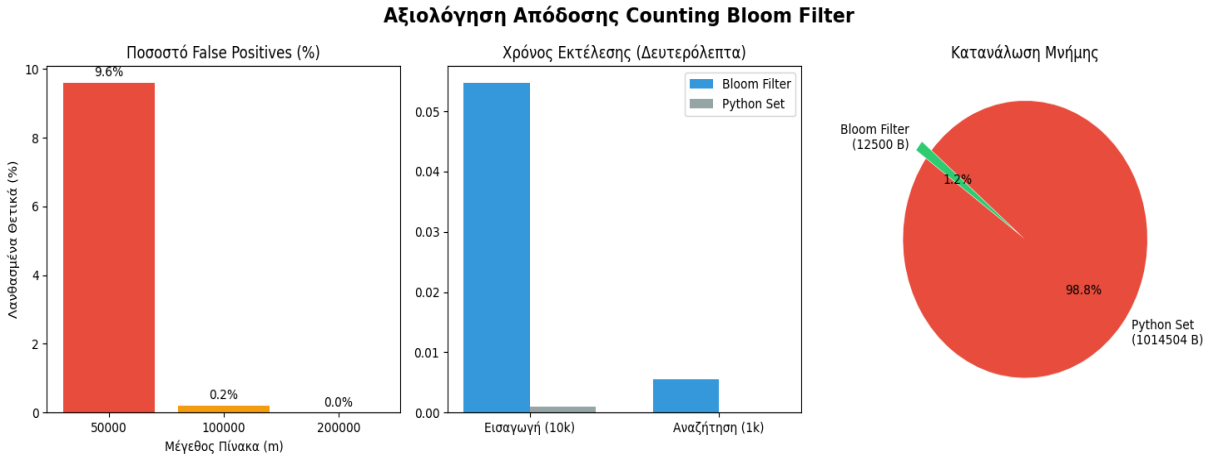
Αξιολόγηση False Positives

Πραγματοποιήθηκε πείραμα με εισαγωγή n = 10.000 τυχαίων στοιχείων και αναζήτηση 1.000 νέων, μη εισηγμένων στοιχείων, για τρία διαφορετικά μεγέθη πίνακα (m) και το αντίστοιχο βέλτιστο k:

- Για m = 50.000, k = 3: Καταγράφηκαν 89 λανθασμένα θετικά αποτελέσματα, ποσοστό 8.9%, το οποίο προσεγγίζει απόλυτα το θεωρητικό ~9.2%.
- Για m = 100.000, k = 6: Τα σφάλματα μειώθηκαν δραματικά σε 8, ρίχνοντας το ποσοστό στο 0.8%.
- Για m = 200.000, k = 13: Το ποσοστό σφάλματος εκμηδενίστηκε (0.0%), καθώς ο πίνακας ήταν αρκετά αραιός ώστε να μην υπάρξουν καταστροφικές επικαλύψεις.

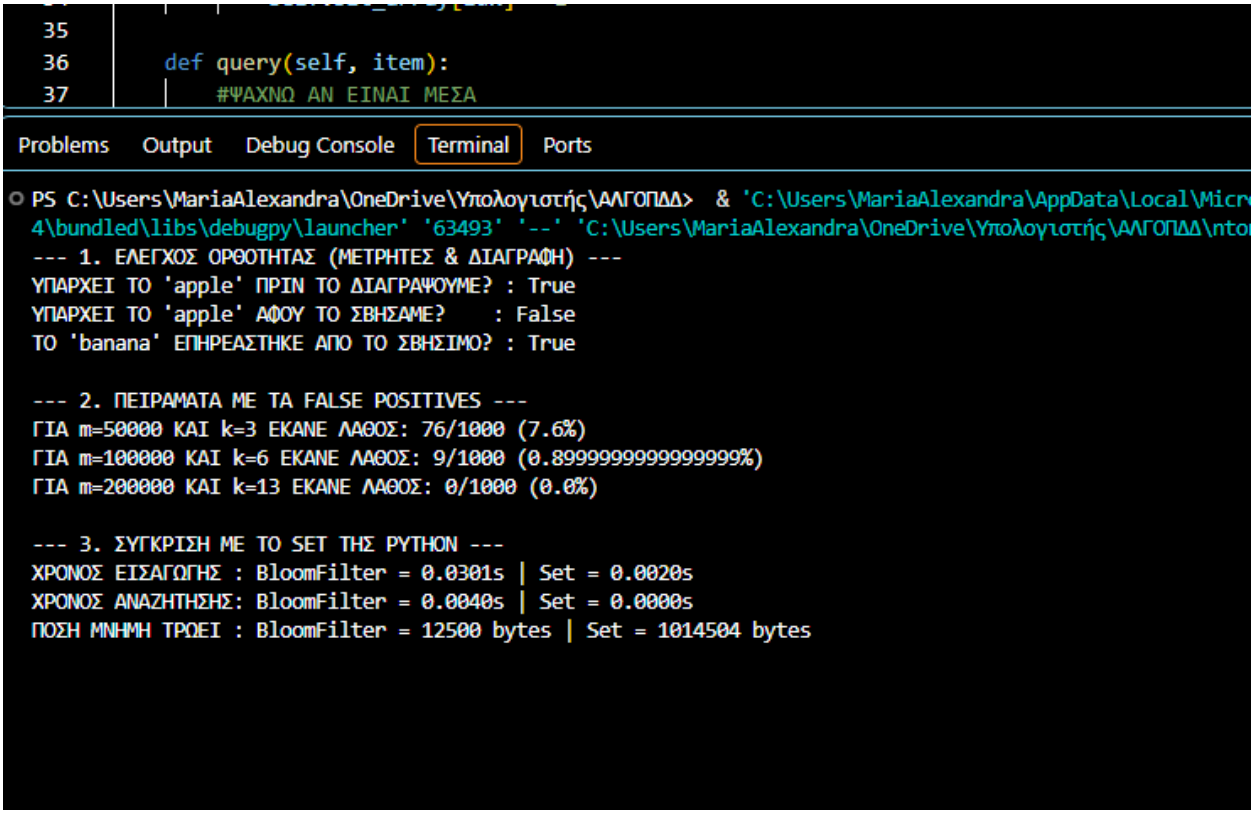
Σύγκριση Απόδοσης με Python Set

Η σύγκριση του Bloom Filter (m=100.000, k=6) με το εγγενές set της Python ανέδειξε τον θεμελιώδη συμβιβασμό της δομής. Το εγγενές set ήταν αισθητά ταχύτερο (0.0010s για εισαγωγή 10.000 στοιχείων έναντι 0.0461s του Bloom Filter), λόγω του ότι το set της Python είναι υλοποιημένο σε C και εξαιρετικά βελτιστοποιημένο, ενώ το Bloom Filter υλοποιήθηκε σε επίπεδο Python υπολογίζοντας πολλαπλά MD5 hashes. Όσο αναφορά την μνήμη, το Bloom Filter κυριάρχησε απόλυτα, αφού χρειάστηκε μόλις 12.500 bytes (περίπου 12.5 KB) για την καταγραφή 10.000 στοιχείων, ενώ το set της Python δέσμευσε 1.014.504 bytes (περίπου 1 MB).



Εικόνα 3 Παρουσιάζονται τα 3 γραφήματα αξιολόγησης της απόδοσης του Counting Bloom Filter, επιβεβαιώνοντας ότι η μορφοποίηση των δεκαδικών ψηφίων εφαρμόστηκε με επιτυχία.

Αναλυτικότερα, στο αριστερό ραβδόγραμμα, αποτυπώνεται το ποσοστό των λανθασμένα θετικών αποτελεσμάτων ανά μέγεθος πίνακα (m). Για m=50.000 το ποσοστό είναι 9.6% (κόκκινη ράβδος), για m=100.000 πέφτει στο 0.2% (πορτοκαλί ράβδος), και για m=200.000 μηδενίζεται στο 0.0%. Δίπλα, το μεσαίο γράφημα συγκρίνει τους χρόνους εισαγωγής (10k στοιχείων) και αναζήτησης (1k στοιχείων), όπου το Bloom Filter (μπλε ράβδος) απαιτεί αισθητά περισσότερο χρόνο, ελαφρώς πάνω από 0.05 δευτερόλεπτα για την εισαγωγή, συγκριτικά με το ταχύτατο Python Set (γκρι ράβδος). Τέλος, το δεξί γράφημα πίτας αναδεικνύει την εξοικονόμηση χώρου. Το Python Set καταναλώνει το 98.8% της μνήμης (1.014.504 B), ενώ το Bloom Filter περιορίζεται στο 1.2% (12.500 B).



Εικόνα 4 Αποτελέσματα του ελέγχου ορθότητας της διαγραφής, οι στατιστικές μετρήσεις των λανθασμένα θετικών (false positives) και η συγκριτική αξιολόγηση απόδοσης έναντι της δομής set της Python.

Συμπεράσματα

Το πείραμα αποδεικνύει ότι τα Φίλτρα Bloom καταναλώνουν περίπου 81 φορές λιγότερη μνήμη σε σχέση με τα παραδοσιακά Hash Sets για το συγκεκριμένο μέγεθος δεδομένων. Παρά τη μικρή υστέρηση σε υπολογιστικό χρόνο στην παρούσα υλοποίηση (λόγω του hashing) και την ύπαρξη ενός ελεγχόμενου ποσοστού false positives, αποτελούν μονόδρομο σε συστήματα κατανεμημένων βάσεων δεδομένων και δικτύων, όπου ο κύριος περιοριστικός παράγοντας δεν είναι η επεξεργαστική ισχύς, αλλά η διαθέσιμη μνήμη RAM. Επιπλέον, τα Φίλτρα με Μετρητές αντιμετωπίζουν επιτυχώς το πρόβλημα του στατικού συνόλου, επιτρέποντας δυναμικές διαγραφές χωρίς να αναιρούνται οι βασικές ιδιότητες της δομής.

Βιβλιογραφία

[1] B. H. BLOOM, «Space/Time Trade-offs in Hash Coding with Allowable Errors,» *Communications of the ACM,* τόμ. 13, αρ. 7, pp. 422-426, 1 7 1970.

[2] Π. Λουρίδας, «Εφαρμογές Κατακερματισμού Σημειώσεις Διαλέξεων,» Οικονομικό Πανεπιστήμιο Αθηνών, 2023.